

⚙ Settings System: Manager, Controller

SettingsController.

- Почти все игры, за исключением самых примитивных, требуют наличие настроек. Где-то настройки очень глобальные и детальные, а где-то попроще. Мне достаточно будет пары параметров.



SettingsController.

- Настройка громкости музыки, громкости эффектов, разрешение экрана, режим окна, сброс к стартовым настройкам, а также ссылка на GITHUB.



DropDown.

```

1 using System.Collections.Generic;
2 using System.Linq;
3 using TMPPro;
4 using UnityEngine;
5 using UnityEngine.UI;
6
7 public class SettingsMenuController : Manager<SettingsMenuController>
8 {
9     [Header("Audio")]
10    [SerializeField] private Slider musicSlider;
11    [SerializeField] private Slider sfxSlider;
12
13    [Header("Resolution")]
14    [SerializeField] private TMP_Dropdown resolutionDropdown;
15    private Resolution[] availableResolutions;
16    private int currentResolutionIndex = 0;
17
18    [Header("CheckBox")]
19    [SerializeField] private Image checkboxImage;
20    [SerializeField] private Sprite checkedSprite;
21    [SerializeField] private Sprite uncheckedSprite;
22    private bool isOnFullScreen = true;
23
24    [Header("Github")]
25    [SerializeField]
26    private string githubUrl =
27    "https://github.com/ffriendlyggghostt";
28
29    protected override void Awake()
30    {
31        base.Awake();
32
33        Screen.fullScreenMode = FullScreenMode.FullScreenWindow;
34        InitResolutions();
35        gameObject.SetActive(false);
36    }
37
38    // В Awake вызывается InitResolutions() - инициализация разрешений экр
39
40    private void Start() => ApplySavedSettings();
41
42    private void Update()
43    {
44        if (Screen.fullScreen != isOnFullScreen)
45        {
46            SyncUI();
47        }
48    }
49
50    public void Hide()
51    {
52        gameObject.SetActive(false);
53        SettingsService.Instance.SaveSettings();
54    }
55
56    public void Show()
57    {
58        gameObject.SetActive(true);
59    }
60
61    public void OnMusicVolumeChanged(float value) =>
62        SettingsService.Instance.SetMusicVolume(value);
63
64    public void OnSFXVolumeChanged(float value) =>
65        SettingsService.Instance.SetSFXVolume(value);
66

```

```

67 // Для ползунков есть OnMusicVolumeChanged(float value)
68 // & OnSFXVolumeChanged(float value) - которые срабатывают при их пере
69 // Они обновляют текущую громкость через SettingsService.
70
71
72 private void InitResolutions()
73 {
74     var allResolutions = Screen.resolutions;
75
76     var filtered = allResolutions
77         .GroupBy(r => new { r.width, r.height })
78         .Select(g => g.OrderByDescending(r => r.refreshRateRatio.v
79         .ToList());
80
81     filtered = filtered
82         .OrderByDescending(r => r.width * r.height)
83         .ToList();
84
85     availableResolutions = filtered.ToArray();
86
87     var options = availableResolutions
88         .Select(r => $"{r.width}x{r.height}")
89         .ToList();
90
91     resolutionDropdown.ClearOptions();
92     resolutionDropdown.AddOptions(options);
93 }
94
95 /*
96 Мы получаем все подходящие для данного экрана разрешения, фильтруем
97 LINQ, чтобы не было повторений под разную герцовку монитора,
98 оставляя только самую высокую герцовку для каждого разрешения,
99 а также сортируем в порядке убывания.
100 После происходит ещё одна сортировка, где мы учитываем общее количес
101 В массив доступных разрешений мы как раз присваиваем полученные резуль
102 После чего в отдельное поле записываем их как string, и закидываем в D
103 */
104
105 public void OnResolutionChanged(int index)
106 {
107     Resolution res = availableResolutions[index];
108     Screen.SetResolution(
109         res.width,
110         res.height,
111         Screen.fullScreen
112     );
113     currentResolutionIndex = index;
114     SettingsService.Instance.SetResolutionChanged(index);
115 }
116
117 /*
118 У нас есть OnResolutionChanged(int index) - который будет вызываться
119 выборе колонки в выпадающем списке. Индекс совпадает с индексами в мас
120 доступных разрешений. Мы просто достаём значение по индексу и устанавли
121 его через Screen.SetResolution.
122
123 К слову, третьим аргументом мы передаём Screen.fullScreen - то есть:
124 изменить разрешение, но не изменять текущий режим окна.
125 Если окно в полноэкранном режиме, то рендер графики уменьшится,
126 но физическая картинка останется такой же, растянувшись (картинка стан
127
128 Игрок может в любое время нажать alt+enter, что сменит режим игры на
129 но эта системная операция никак не уведомит об этом мой скрипт.
130 Есть OnApplicationFocus(bool) - но он работает слегка не так, как хоте
131 Ранее я вызывал SyncUI() в методе Show(), при открытии меню. Однако пр
132 меню это никак не обновляло текущую галочку FullScreen. В итоге, кто-т
133 убедил добавить проверку текущего поля isOnFullScreen и Screen.fullScr

```

```

134 // Update.
135 */
136     private void SyncUI()
137     {
138         isOnFullScreen = Screen.fullScreen;
139         SettingsService.Instance.SetIsFullScreen( isOnFullScreen );
140         RefreshFullscreenUI();
141     }
142
143     private void RefreshFullscreenUI()
144     {
145         checkboxImage.sprite =
146             isOnFullScreen ? checkedSprite : uncheckedSprite;
147     }
148
149     public void DefaultSettings()
150     {
151         musicSlider.value = 1f;
152         sfxSlider.value = 1f;
153
154         isOnFullScreen = true;
155         RefreshFullscreenUI();
156         Screen.SetResolution(
157             availableResolutions[0].width,
158             availableResolutions[0].height,
159             true
160         );
161
162         resolutionDropdown.value = 0;
163         currentResolutionIndex = 0;
164         resolutionDropdown.RefreshShownValue();
165
166         SettingsService.Instance.SetDefaultSettings();
167     }
168
169     // DefaultSettings() - сбрасывает все настройки на стандартные.
170     // Заодно и сообщает SettingsService, что настройки сбросились.
171
172     public void FullScreenToggle()
173     {
174         isOnFullScreen = !isOnFullScreen;
175         RefreshFullscreenUI();
176         Screen.fullScreen = isOnFullScreen;
177         SettingsService.Instance.SetIsFullScreen( isOnFullScreen );
178     }
179
180     // FullScreenToggle - вызывается при нажатии на чекбокс.
181     // Меняет спрайт и изменяет режим окна.
182
183     public void GitHubButton() => Application.OpenURL(githubUrl);
184
185     public void ApplySavedSettings()
186     {
187         var service = SettingsService.Instance;
188
189         musicSlider.value = service.MusicVolume;
190         sfxSlider.value = service.SFXVolume;
191
192         isOnFullScreen = service.IsFullScreen;
193         currentResolutionIndex = service.ResolutionIndex;
194
195         RefreshFullscreenUI();
196
197         currentResolutionIndex = Mathf.Clamp(
198             service.ResolutionIndex,
199             0,
200             availableResolutions.Length - 1

```

```

201     );
202
203     Resolution res = availableResolutions[currentResolutionIndex];
204     Screen.SetResolution(res.width, res.height, isOnFullScreen);
205
206     resolutionDropdown.SetValueWithoutNotify(currentResolutionIndex);
207     resolutionDropdown.RefreshShownValue();
208 }
209
210 /*
211  ApplySavedSettings - запишем ссылку на синглтон сервис.
212  Далее в наши поля присваиваем поля из сервиса, обновляем UI.
213  SetValueWithoutNotify, при установке значения в выпадающем списке разрешит избежать вызова метода OnResolutionChanged(), чтобы не совершить
214  лишних вызовов кода с тяжелой функцией Screen.SetResolution(), которая вызывалась повторно, хотя нам это не нужно.
215
216  Интересный момент. currentResolutionIndex = Mathf.Clamp
217  ( service.ResolutionIndex, 0, availableResolutions.Length - 1);
218  → защита разрешений. Если игрок с 4K монитором скинет сохранение товара
219  у которого монитор 1366x768 - то у второго игрока контроллер бы попытался
220  присвоить индекс (например 15) из массива длиной 4, возможно захватив
221  оперативной памяти. А позже попытается установить расширение применяя
222  валидные данные, что вызвало бы ошибки и баги. В нашем случае мы указываем
223  передаваемое поле, минимум - индекс 0, и максимум - доступные разрешения
224  данного экрана. Если даже сохранения первого окажутся у второго,
225  то будет так: (15, 0, 4) и операция выдаст нам 4, как максимальный
226  возможный результат. Б - Безопасность.
227 */
228
229 }
230

```

SettingsService.

- Изначально всю информацию о настройках (сами данные) и запрос в SaveManager осуществлял контроллер. На самом деле это крайне не верно, если соблюдать архитектурный порядок. UI и данные должны лежать раздельно, более того сама логика - тоже должна лежать отдельно. В моём случае есть UI (который управляется Unity) → есть Controller, который данные не хранит, он реагирует на UI и отправляет информацию в Service. По сути - он является связывающим мостом. Service - содержит все данные настроек, отдаёт их по запросу контроллера, и сохраняет через SaveManager - также по запросу контроллера.

Сие паттерн называется MVP. Model - View - Presenter. Он отличается от MVC, и в Unity встречается чаще. Данные лежат отдельно, UI лежит отдельно, Presenter - контроллер, знает о UI и вручную их обновляет. Поток выглядит вот так: Input → Presenter → Model → Presenter → View.

В MVC View чаще сама наблюдает за Model. А поток выглядит так: Input → Controller → Model changed → View. UI зависит от данных.

```

1 using System;
2 using UnityEngine;
3
4 public class SettingsService : Manager<SettingsService>

```

```

5 {
6     public float MusicVolume { get; private set; } = 1f;
7     public float SFXVolume { get; private set; } = 1f;
8     public bool IsFullScreen { get; private set; } = true;
9     public int ResolutionIndex { get; private set; } = 0;
10
11     public event Action<float> OnMusicVolumeChanged;
12     public event Action<float> OnSFXVolumeChanged;
13     public event Action<bool> OnIsFullScreenChanged;
14     public event Action<int> OnResolutionIndexChanged;
15
16     private void Start()
17     {
18         InitializeFromSave();
19     }
20     private void InitializeFromSave()
21     {
22         var data = SaveManager.Instance.Data.settings;
23
24         MusicVolume = data.musicVolume;
25         SFXVolume = data.sfxVolume;
26         IsFullScreen = data.isFullScreen;
27         ResolutionIndex = data.resolutionIndex;
28     }
29
30     // InitializeFromSave - на старте подгружаем данные из SaveManager.
31     // Расфасовываем по полям.
32
33     public void SetMusicVolume(float value)
34     {
35         if (Mathf.Abs(MusicVolume - value) < 0.001f) return;
36
37         MusicVolume = value;
38         OnMusicVolumeChanged?.Invoke(value);
39     }
40
41     public void SetSFXVolume(float value)
42     {
43         if (Mathf.Abs(SFXVolume - value) < 0.001f) return;
44
45         SFXVolume = value;
46         OnSFXVolumeChanged?.Invoke(value);
47     }
48
49     /*
50     SetMusicVolume() \ SetSFXVolume() - меняют данные громкости и вызывают
51     о том, что данные изменились. Слушать это событие будет AudioManager.
52     К слову, мы делаем проверку что пришедшее число:
53     Mathf.Abs(MusicVolume - value) < 0.001f (модуль пришедшего числа отлич
54     от предыдущего более чем 0.001, чтобы избежать лишних вызовов).
55     => Epsilon comparison.
56     */
57
58     public void SetIsFullScreen(bool value)
59     {
60         if (IsFullScreen == value) return;
61         IsFullScreen = value;
62         OnIsFullScreenChanged?.Invoke(value);
63     }
64
65     // SetIsFullScreen() - меняет текущее значение поля fullScreen.
66
67     public void SetResolutionChanged(int value)
68     {
69         if (ResolutionIndex == value) return;
70         ResolutionIndex = value;
71         OnResolutionIndexChanged?.Invoke(value);

```

```

72     }
73
74     // SetResolutionChanged() - меняет текущее значение поля индекса разреш
75
76     public void SetDefaultSettings()
77     {
78         SetSFXVolume(1f);
79         SetMusicVolume(1f);
80         SetIsFullScreen(true);
81         SetResolutionChanged(0);
82     }
83
84     // SetDefaultSettings() - сбрасывает настройки к стандартным.
85
86     public void SaveSettings()
87     {
88         var data = SaveManager.Instance.Data.settings;
89
90         data.musicVolume = MusicVolume;
91         data.sfxVolume = SFXVolume;
92         data.isFullScreen = IsFullScreen;
93         data.resolutionIndex = ResolutionIndex;
94
95         SaveManager.Instance.Save();
96     }
97     // SaveSettings() - присваиваем текущие данные в Data SaveManager'a,
98     // и вызываем сохранение.
99 }

```

📝 Настройки работают корректно, отображаются и изменяются в реальном времени. Всё подгружается при старте игры и всё сохраняется в json документ. Настройки довольно таки простые, но систему можно легко расширить и добавить множество других настроек. Мне этих для данного проекта более чем достаточно.

✅ 28.05.2026 23:49 - Не знаю... Кто бы это ни читал... Спасибо, что-ли... 😊
 Сложно представить, что кто-то будет возится с чтением документации к какой-то по-наме pet-project игре разраба из глубинки. Но если вы это читаете... Пускай Госпожа Удача будет на вашей стороне на этой жизненной тропе. Я всего лишь учусь документировать свою работу и заодно повторяю какие-то аспекты. Пу-пу-пу...